

Advanced Cybersecurity Phishing Detection Platform

Complete Technical Documentation

Generated: August 17, 2025
Version: 1.0

Table of Contents

1. Project Overview	2
2. System Architecture	3
3. Database Schema	4
4. File Structure & Connections	5
5. Python Dependencies & Imports	7
6. Installation Guide	9
7. Running Commands	11
8. API Endpoints	12
9. Testing & Validation	14
10. Troubleshooting	15

1. Project Overview

The Advanced Cybersecurity Phishing Detection Platform is a comprehensive email analysis system designed to detect phishing attempts and malicious content using machine learning and pattern recognition techniques. The platform analyzes email files in multiple formats (.eml, .msg, .txt) and provides real-time threat scoring, batch processing capabilities, and detailed reporting.

Key Features:

- HTML segment analysis with line number tracking
- Email header analysis including x-ms-exchange-crosstenant headers
- PostgreSQL database for storing analysis results and statistics
- Segment classification (single/multiple line, HTML types)
- Real-time dashboard with threat analytics
- Command-line tools for database statistics
- RESTful API with comprehensive endpoints
- Responsive web interface with dark/light themes

2. System Architecture

Backend Architecture:

The system follows a modular Flask-based REST API architecture with clear separation of concerns:

Component	Technology	Purpose
Web Framework	Flask	HTTP request handling and API endpoints
Database	PostgreSQL	Persistent storage of analysis results
Email Parser	Python email library	Multi-format email parsing
HTML Analyzer	Custom analyzer	HTML segment extraction and analysis
Pattern Detection	RegEx + ML	Threat pattern recognition
Frontend	HTML/CSS/JavaScript	User interface and dashboard

3. Database Schema

The system uses PostgreSQL with the following tables:

Table Name	Purpose	Key Fields
email_analyses	Main analysis results	analysis_id, filename, threat_score, threat_level
email_headers	Header analysis	analysis_id, header_name, is_suspicious
html_segments	HTML segment details	analysis_id, segment_type, is_single_line
threat_detections	Individual threats	threat_category, threat_pattern, line_number
analysis_failures	Failed analyses	filename, failure_reason, failure_type
threat_statistics	Aggregated stats	threat_category, occurrence_count

4. File Structure & Connections

Project Directory Structure:

```
phishing-detection-platform/
├── python/
│   ├── simple_app.py # Main Flask application
│   ├── email_html_analyzer.py # Email analysis engine
│   ├── database_schema.py # Database operations
│   ├── database_stats_tool.py # Command-line statistics
│   ├── config.py # Configuration settings
│   └── static/
│       ├── index.html # Web interface
│       ├── test_phishing_email.eml # Basic test sample
│       ├── test_safe_email.eml # Safe email sample
│       ├── test_enhanced_phishing_email.eml # Advanced test sample
│       ├── uploads/ # Temporary file storage
│       └── logs/ # Application logs
```

File Connections & Dependencies:

File	Imports/Uses	Purpose
simple_app.py	email_html_analyzer, database_schema	Main app importing analysis and DB modules
email_html_analyzer.py	email, html2text, re	Email parsing and HTML analysis
database_schema.py	psycopg2, os	PostgreSQL operations
database_stats_tool.py	database_schema, email_html_analyzer	CLI tool using DB and analyzer
index.html	Chart.js, Font Awesome	Frontend using charting and icons

5. Python Dependencies & Imports

Required Python Packages:

Package	Version	Purpose	Used In
Flask	Latest	Web framework for API	simple_app.py
Flask-CORS	Latest	Cross-origin resource sharing	simple_app.py
psycopg2-binary	Latest	PostgreSQL database adapter	database_schema.py
html2text	Latest	HTML to text conversion	email_html_analyzer.py
email-validator	Latest	Email validation	email_html_analyzer.py
reportlab	Latest	PDF generation	generate_documentation.py
werkzeug	Latest	Secure filename handling	simple_app.py
numpy	Latest	Numerical operations	Optional for ML
pandas	Latest	Data manipulation	Optional for analytics
scikit-learn	Latest	Machine learning	Optional for ML models

Key Import Explanations:

• email: Built-in Python library for parsing email files (.eml format) • html2text: Converts HTML content to plain text for analysis • psycopg2: PostgreSQL adapter for Python database operations • flask: Web framework for creating REST API endpoints • werkzeug: Provides secure_filename() for safe file handling • os: Operating system interface for environment variables • re: Regular expressions for pattern matching in threat detection • datetime: Date and time handling for timestamps • uuid: Unique identifier generation for analysis sessions • logging: Application logging and error tracking

6. Installation Guide

System Requirements:

- Python 3.8 or higher
- PostgreSQL 12 or higher
- 4GB RAM minimum
- 10GB disk space

Step 1: Install PostgreSQL

```
# Ubuntu/Debian: sudo apt update sudo apt install postgresql
postgresql-contrib # macOS (using Homebrew): brew install postgresql
brew services start postgresql # Windows: Download from
https://www.postgresql.org/download/windows/
```

Step 2: Create Database

```
# Connect to PostgreSQL sudo -u postgres psql # Create database and
user CREATE DATABASE phishing_detection; CREATE USER phishing_user
WITH PASSWORD 'secure_password'; GRANT ALL PRIVILEGES ON DATABASE
phishing_detection TO phishing_user; \q
```

Step 3: Set Environment Variables

```
# Linux/macOS (.bashrc or .zshrc): export DATABASE_URL="postgresql://
phishing_user:secure_password@localhost:5432/phishing_detection" #
Windows (Command Prompt): set DATABASE_URL=postgresql://phishing_user
:secure_password@localhost:5432/phishing_detection # Windows
(PowerShell): $env:DATABASE_URL="postgresql://phishing_user:secure_pa
ssword@localhost:5432/phishing_detection"
```

Step 4: Install Python Dependencies

```
# Create virtual environment (recommended) python -m venv phishing_env
# Activate virtual environment # Linux/macOS: source
phishing_env/bin/activate # Windows: phishing_env\Scripts\activate #
Install required packages pip install flask flask-cors psycpg2-binary
html2text email-validator pip install reportlab werkzeug numpy pandas
scikit-learn
```

Step 5: Download Project Files

```
# Create project directory mkdir phishing-detection-platform cd
phishing-detection-platform # Copy all Python files to python/
directory # Copy test .eml files to root directory # Ensure proper
file structure as shown in section 4
```

7. Running Commands

Start the Main Application:

```
# Navigate to python directory cd python # Start Flask application
python simple_app.py # Application will start on http://localhost:5000
# Access web interface at http://localhost:5000
```

Database Statistics Tool:

```
# View all statistics python database_stats_tool.py all # View threat
statistics only python database_stats_tool.py threats # View failure
statistics python database_stats_tool.py failures # View dashboard
statistics python database_stats_tool.py dashboard # Test analyzer
with sample files python database_stats_tool.py test
```

Testing with Sample Files:

The platform includes three test email files: • test_phishing_email.eml - Basic phishing patterns • test_safe_email.eml - Clean email sample • test_enhanced_phishing_email.eml - Advanced threats with headers You can download these from the web interface or analyze them directly.

8. API Endpoints

Method	Endpoint	Purpose	Parameters
GET	/	Main web interface	None
POST	/api/upload	Upload email for analysis	file (multipart)
GET	/api/dashboard-stats	Get dashboard statistics	None
GET	/api/threat-statistics	Get threat occurrence stats	None
GET	/api/failure-statistics	Get analysis failure stats	None
POST	/api/detailed-report	Get detailed HTML report	file (.eml)
POST	/api/analyze-text	Analyze text content	content (JSON)
GET	/test_*.eml	Download test samples	None

Example API Usage:

```
# Upload email file using curl curl -X POST -F  
"file=@test_phishing_email.eml" http://localhost:5000/api/upload # Get  
dashboard statistics curl http://localhost:5000/api/dashboard-stats #  
Analyze text content curl -X POST -H "Content-Type: application/json"  
\ -d '{"content": "Click here to verify your account"}' \  
http://localhost:5000/api/analyze-text
```

9. Testing & Validation

Threat Detection Categories:

- Urgent Actions (expires in 24 hours, act now)
- Credential Harvesting (enter your password, confirm login)
- Financial Threats (unauthorized transaction, suspicious activity)
- Microsoft Exchange (crosstenant authentication, office 365)
- Suspicious Codes (specific patterns like XFCjGeORft8x7OI)
- Header Analysis (x-ms-exchange-crosstenant headers)

Validation Results:

The enhanced test email successfully detects: ✓ 15 different threat patterns ✓ 8 suspicious email headers ✓ 41 HTML segments analyzed ✓ 100/100 threat score (HIGH risk) ✓ Proper database storage and statistics

10. Troubleshooting

Common Issues and Solutions:

Issue	Cause	Solution
Database connection error	Wrong DATABASE_URL	Check connection string and credentials
Import errors	Missing packages	Install all required dependencies
File upload fails	Large file size	Check file size limits in config
UUID errors	Old database schema	Drop and recreate tables
Port already in use	Flask port conflict	Change port in simple_app.py
Permission denied	File access rights	Check file permissions

Debug Commands:

```
# Check database connection python -c "from database_schema import DatabaseManager; db = DatabaseManager(); print('DB OK')" # Test email analyzer python -c "from email_html_analyzer import EmailHTMLAnalyzer; analyzer = EmailHTMLAnalyzer(); print('Analyzer OK')" # View application logs tail -f logs/application.log # Check database tables psql -d phishing_detection -c "\dt"
```

End of Documentation

Generated on August 17, 2025 at 01:41 PM